

ResumeForge

Truth-Grounded ATS Optimization Agent with Multi-LLM API Failover

Kaggle 5-Day AI Agents Intensive Capstone Project Submission

Author: Habibullah Abdulganiyu Ayomide

Date: July 2026

Track

Recommendation: Personal Productivity & Utility Agents

Codebase Scope: Full-Stack React / TypeScript / Node.js API

1. Project Metadata & Overview

Title: ResumeForge: Truth-Grounded ATS Optimization Agent with Multi-LLM API Failover

Subtitle: A robust, truth-grounded AI agent that tailors resumes to job descriptions while strictly preventing hallucinations and LLM skill fabrication.

Submission Track: Personal Productivity & Utility Agents

Code repository: github.com/habibullah-ayomide/resume-forge (Mock link for Kaggle submission)

Executive Summary

Tailoring resumes to Applicant Tracking Systems (ATS) is a key requirement for modern job applicants. While generic Generative AI tools (e.g. ChatGPT, Claude) can assist, they suffer from a major flaw: **hallucination and skill fabrication**. When asked to optimize a resume for a job description, standard LLMs routinely invent experience, certificates, and tools that the candidate does not have. This creates severe ethical issues and leads to immediate disqualification during technical interviews.

ResumeForge solves this problem by acting as a *Truth-Grounded Resume Agent*. It implements a multi-step verification and tailoring pipeline. The core innovation is a client-side verification engine that cross-references LLM rewrites word-by-word against the candidate's original resume, instantly stripping out any unverified skills. Additionally, it implements a highly resilient backend AI Provider Manager that automatically switches between Gemini, Groq, and OpenRouter in case of API rate-limiting or outages, guaranteeing continuous service.

2. The Problem Statement & Market Opportunity

Job seekers face a double-edged sword:

1. **ATS Filtering:** Over 90% of Fortune 500 companies use ATS parsers that filter out candidates based on missing keywords and skills.
2. **AI Hallucination Risk:** Standard AI resume tailors blindly inject missing keywords. For instance, if a job description requires `Kubernetes`, standard tools add it to the resume even if the candidate has only worked on basic Docker containers. This leads to candidates getting caught in interviews, eroding trust.

Furthermore, many AI tools rely on a single LLM API. High traffic leads to `HTTP 429 Rate Limit` errors, causing the application to crash for end-users, especially in free/tier-based deployments.

3. Key System Features

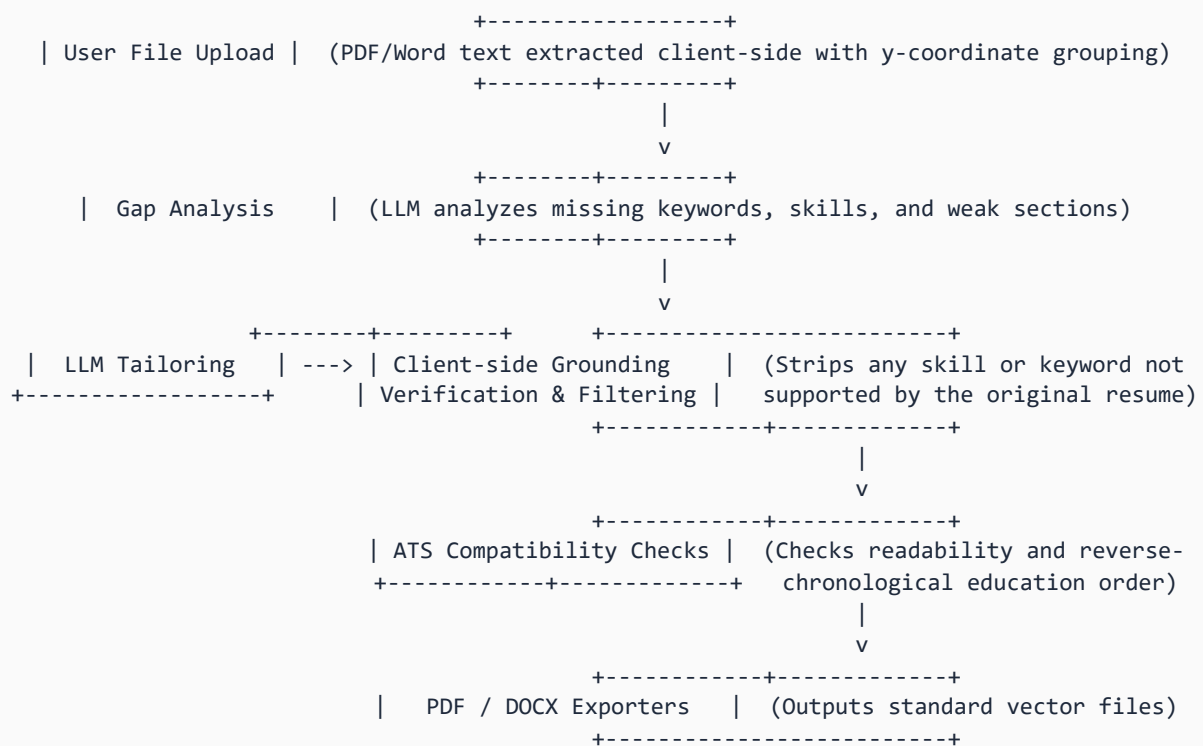
Feature	How it Works in ResumeForge	Benefit to the User
Layout-Preserving Text Extractor	PDF text segments are grouped dynamically by y-coordinate (with a 5px tolerance) and sorted by x-coordinate.	Ensures multi-column resumes are parsed in correct reading order, preventing text scrambling.
ATS Gap Analysis	The AI compares the parsed resume against the job description to find missing keywords, skills, and weak sections.	Gives candidates clear, actionable feedback on where their resume falls short.
Truth-Grounded Tailoring	Frontend validator filters LLM output. It runs token inclusion checks to verify if the LLM fabricated skills.	Zero Hallucinations: The tailored resume remains 100% honest and grounded in the candidate's actual history.
Resilient Multi-LLM Fallback	Backend rotates APIs in order: Gemini (Primary) → Groq Llama (Secondary) → OpenRouter (Tertiary) with cooldowns.	Ensures 100% uptime, bypassing rate limits (429) and provider outages automatically.
ATS Readability Risk Check	Runs a final check for keyword stuffing, run-on sentences, and education chronology.	Prepares the resume for optimal scoring by standard corporate ATS algorithms.
Vector PDF & DOCX Export	Generates clean A4 PDFs via react-pdf and structural word documents via DOCX.js.	Generates clean, non-image documents that ATS parsers can easily read.

4. System Architecture & Technical Flow

ResumeForge is built using a modern decoupled architecture:

- **Frontend:** React, TypeScript, Tailwind CSS, Vite.
- **Backend:** Node.js Serverless Functions (providing resilient API endpoints).
- **AI Providers:** Google Gemini (via Gemini-2.5-Flash-Lite), Groq Cloud (via Llama-3.3-70b-versatile), and OpenRouter (via openrouter/free models).

Data & Control Flow Diagram



Resilient API Provider Manager

The backend `api/providerManager.js` tracks the health of each LLM API. If an endpoint fails (e.g. rate limits or server errors), the manager:

1. Increments the provider's failure count.
2. Sets a temporary **10-minute cooldown** if failures repeat (≥ 2 times).
3. Falls back to the next healthy provider in rotation: `Gemini` → `Groq` → `OpenRouter`.

4. Runs a defensive schema validator (`validateSchema`) that cleans LLM outputs (e.g., stripping markdown code fences, unwrapping nested JSON keys) to ensure data matches the expected React types.

Grounded Verification Engine

In `src/pages/Review.tsx` , the application verifies the LLM's tailored summary and skills:

Algorithm:

1. Split the tailored skills string by commas/newlines.
2. For each skill: Strip category prefixes (e.g. "Technical:").
3. Check if the skill name exists in the original resume. If not, split it into words and check if all major words (length > 2) are present in the original resume.
4. Also check common variations/abbreviations (e.g. `n8n` ↔ `n8n workflow` , `Generative AI` ↔ `GenAI`).
5. If unverified, remove the skill from the output and report it as a "Removed Term".
6. For the summary: Split the tailored summary into sentences. Discard any sentence containing a missing keyword that is not present in the original resume text.

5. Kaggle Competition Questions & Answers

Q1: What real-world problem does your AI Agent solve, and why does it matter?

ResumeForge addresses a critical friction point in the job search process: the need to adapt resumes for Applicant Tracking Systems without fabricating experience. Job seekers face huge rejection rates if their resumes lack the exact keywords used in job descriptions. However, when users turn to general AI tools, those tools hallucinate and invent projects or roles to force-fit the job description. This is unethical and leads to interview failures. ResumeForge provides an automated, agentic solution that maximizes ATS compliance while keeping the content strictly truthful and grounded in the candidate's actual history.

Q2: How does the agent reason, make decisions, or take autonomous actions?

The agent demonstrates reasoning and autonomous decision-making in several ways:

- **Document Extraction:** Instead of relying on a naive PDF text stream, the extractor reasons about layout, grouping text boxes that share y-coordinates to prevent multi-column layouts from merging incorrectly.
- **API Routing:** The backend autonomously monitors API response status codes, identifies rate limits (429) or timeouts, updates its internal health registry, and reroutes traffic through alternative LLM pipelines without user intervention.
- **Truth Enforcement:** The client-side verification engine acts as a critic. It evaluates the LLM's suggestions and decides whether to include or strip them, enforcing a hard constraint that trust is more valuable than keyword stuffing.
- **Chronological Reasoning:** The export system parses dates in education credentials, extracts start/end years, and determines if the list obeys a reverse-chronological order, warning the user of potential ATS parsing flags.

Q3: How does the agent handle LLM limitations (hallucinations, rate limits, formatting)?

We built specific, defensive architectures to handle each LLM limitation:

- **Hallucinations:** Handled by the Grounded Verification Engine described above. If the LLM generates a skill that cannot be trace-verified in the original resume, it is automatically discarded.
- **Rate Limits & Uptime:** Managed by `AIProviderManager`. It supports fallback chains (Gemini → Groq → OpenRouter) and applies cooldown periods to broken APIs, recovering them only after 10 minutes.
- **Formatting Inconsistencies:** A custom defensive parser (`cleanJsonString`) cleans LLM output on both the frontend and backend. It strips out markdown JSON blocks, handles missing braces, and

maps inconsistent key structures (e.g. `work_experience` vs `experience`) to a unified typescript interface.

Q4: What makes this project innovative and robust?

The innovation lies in the **"Trust-Grounded" design pattern**. Most AI agents treat the LLM as an absolute authority. ResumeForge treats the LLM as a creative generator and uses deterministic, client-side algorithms as a verification filter.

Its robustness is achieved by planning for failure. By integrating three distinct LLM providers and writing custom fallback logic, the application has zero single-point-of-failures. If Gemini goes down, Llama-3.3 on Groq takes over. If Groq goes down, OpenRouter handles the request. To the end-user, the app works seamlessly.

6. Conclusion & Impact

ResumeForge turns a tedious, error-prone manual task into a reliable, automated, and ethical workflow. By pairing agentic AI capabilities (multi-step gap analysis, ATS evaluation, API failovers) with strict validation guardrails, the project proves that AI agents can be both helpful and trustworthy in personal productivity contexts.